Gods Eye Command Centre
Technical Product Specification and Architecture Document
Palatial Pool Services Pty Ltd
Engineered by Kieran Goldsmith
Classification: Proprietary & Confidential

Table of Contents

1. [Introduction](#introduction)
2. [System Overview](#system-overview)
3. [Architecture and Data Flow](#architecture-and-data-flow)
4. [Core Functional Components](#core-functional-components)
5. [Technology Stack and Infrastructure](#technology-stack-and-infrastructure)
6. [Security Architecture](#security-architecture)
7. [Data Formats and Integration](#data-formats-and-integration)
8. [Operational Monitoring and Reliability](#operational-monitoring-and-reliability)
9. [Compliance and Legal Considerations](#compliance-and-legal-considerations)
10. [Appendices](#appendices)

---

1. Introduction

1.1 Purpose
The Gods Eye Command Centre (GECC) serves as the centralized operational control system supporting Palatial Pool Services Pty Ltd’s entire technology ecosystem. Designed with principles inspired by nuclear power station control rooms, GECC delivers a unified, real-time operational view across all business domains, including customer management, field operations, financial tracking, asset status monitoring, communications, and competitive intelligence.

This document provides a comprehensive technical overview of the GECC, detailing system architecture, data flows, security protocols, infrastructure specifications, and integration methodologies. It is intended for engineers, prospective buyers, and investors seeking an in-depth understanding of the technology underpinning Palatial Pools’ proprietary platform.

1.2 Scope

This document covers the software architecture, data processing pipelines, security frameworks, infrastructure environment, and operational procedures of GECC. It excludes hardware design specifics of field equipment and external third-party interfaces beyond documented API endpoints.

---

2. System Overview

2.1 Functional Description
GECC aggregates and correlates real-time data from disparate systems within the Palatial Pools ecosystem to provide a "single pane of glass" dashboard accessible by authorized operators. The system ingests data streams from IoT devices, mobile field applications, CRM systems, billing and payment gateways, and external market intelligence sources.

Key capabilities include:
- Real-time dashboard visualization with custom widgets
- Role-based multi-user concurrent access
- Automated alerting and workflow orchestration based on configurable business rules
- Audit logging and session management for compliance and traceability

2.2 Design Principles
The system is designed to meet the following engineering principles:

Principle	Description
Reliability	99.9% uptime with failover and health monitoring
Scalability	Microservices architecture supports horizontal scaling
Security	Multi-layered encryption, access control, and audit trails
Extensibility	Modular design facilitates rapid feature addition
Compliance	Adherence to Australian Privacy Act 1988 and industry standards
Real-time Processing	Event-driven architecture with sub-second latency

---

3. Architecture and Data Flow

```
### 3.1 High-Level Architecture Diagram

```mermaid
graph TD
    subgraph Data Sources
        IoT[Pool IoT Devices]
        MobileApp[Mobile Field App]
        CRM[Customer Relationship Management System]
        Billing[Billing & Payments System]
        MarketIntel[Competitive Intelligence Feeds]
    end

    subgraph Gods Eye Command Centre
        EventBus[WebSocket Event Bus]
        Ingest[Data Ingestion Layer]
        Processor[Real-Time Processing & Rules Engine]
        DB[Operational Data Store (PostgreSQL)]
        Dashboard[Dashboard & UI Layer]
        Workflow[Automated Workflow Engine]
        Auth[Authentication & Access Control Module]
        Audit[Audit Logging Service]
    end

    DataSources -->|Real-Time Data Streams| EventBus
    EventBus --> Ingest --> Processor --> DB
    Processor --> Workflow
    DB --> Dashboard
    Auth --> Dashboard
    Workflow --> EventBus
    Dashboard --> Operators[Operator Clients]

    Operators --> Auth
    Operators --> Dashboard
    Processor --> Audit
```

3.2 Data Flow Description

1. **Data Acquisition:**
    - IoT devices deployed in pools transmit sensor readings (e.g., pH, chlorine levels, temperature) over IEEE 802.11 Wi-Fi or LTE networks to the ingestion API via secure MQTT or HTTPS protocols.
```

- Mobile field applications push updates on service tasks, equipment status, and customer interactions via RESTful API endpoints and WebSocket connections.
- CRM and billing systems provide customer and financial data through scheduled batch exports and live API integrations.
- Market intelligence feeds supply competitive and environmental data through third-party API subscriptions.

2. **Event Bus & Ingestion:**

- The WebSocket-based event bus serves as a message broker implementing the Publish-Subscribe pattern (Pub/Sub). It supports asynchronous, low-latency data transmission adhering to WebSocket RFC 6455 standards.
- Event payloads are JSON-encoded, schema-validated using JSON Schema Draft 7, and include metadata such as timestamps (ISO 8601), source ID, and event type.

3. **Processing & Rules Engine:**

- Incoming events are processed in real time by the rules engine implemented using a combination of Node.js microservices and Apache Kafka Streams for stateful stream processing.
- Business rules are configurable via a domain-specific language (DSL) supporting logical expressions, temporal predicates, and action triggers.
- Example rule: Upon detection of a missed payment event, the system triggers an SMS reminder, flags the customer record, and modifies scheduling priority accordingly.

4. **Storage & Query:**

- A PostgreSQL relational database serves as the operational data store, optimized for complex queries and transactional integrity.
- Time-series data from IoT devices is additionally stored in an InfluxDB instance to support trending and analytics.

5. **User Interface & Access:**

- The dashboard UI is a React.js single-page application (SPA) communicating with backend RESTful APIs and WebSocket endpoints for

real-time updates.

- Widgets are customizable per operator role, with alert thresholds and visualizations configurable via drag-and-drop interfaces.

---

4. Core Functional Components

4.1 Dashboard and Visualization

- **Widget Framework:** Modular widgets render KPIs, graphs, maps, and alerts using D3.js and Mapbox GL JS.
- **Customization:** Users can create personalized layouts saved per operator profile.
- **Alerting:** Threshold breaches trigger visual cues and optional notifications (SMS, email).

4.2 Rules Engine and Workflow Automation

- **Rule Definition Language:** Supports Boolean logic, arithmetic operations, and event correlation.
- **Workflow Orchestration:** Integrates with Twilio API for SMS, email services (SMTP), and internal APIs for task assignment and priority changes.
- **Error Handling:** Failures in workflows are logged and raise secondary alerts for manual intervention.

4.3 Access Control and Session Management

- **Authentication:** Multi-factor authentication (MFA) implemented with TOTP (RFC 6238) and FIDO2/WebAuthn options.
- **Authorization:** Role-based access control (RBAC) enforced at API gateway and UI layers using OAuth 2.0 scopes.
- **Session Policies:** Idle session timeouts configurable; concurrent session limits enforced per operator.

4.4 Audit and Compliance Logging

- Immutable logs capture every operator action, system event, and data access with timestamps and operator identifiers.
- Logs are stored encrypted and retained per Australian regulatory requirements.

---

5. Technology Stack and Infrastructure

Layer	Technology / Standard	Description
Hosting	DigitalOcean Sydney (DO-SYD1)	Primary cloud region for data sovereignty and low latency
Containerization	Docker	Microservices containerization for portability and deployment consistency
Orchestration	Kubernetes (k8s)	Automated deployment, scaling, and management of containers
Messaging/Event Bus	WebSocket (RFC 6455), Apache Kafka Streams	Real-time, bi-directional communication and event processing
Backend	Node.js, Express.js	REST API and microservices implementation
Databases	PostgreSQL (v13), InfluxDB	Relational and time-series data storage
Frontend	React.js, Redux	Single-page application with state management
Security	TLS 1.3 (RFC 8446), AES-256 (FIPS 197)	Encryption protocols for data in transit and at rest
Authentication	OAuth 2.0, OpenID Connect, TOTP (RFC 6238)	Secure authentication and authorization
Monitoring	Prometheus, Grafana	System and application performance monitoring
Backup	Automated daily backups, 30-day retention	Data backups with offsite replication

---

6. Security Architecture

6.1 Authentication and Authorization

- **Multi-Factor Authentication (MFA):** Enforced for all operator logins using Time-Based One-Time Passwords (TOTP) compliant with RFC 6238. Optional hardware token support via FIDO2/WebAuthn.
- **Role-Based Access Control (RBAC):** Access permissions granulated down to API endpoint and UI widget level, configurable per user role.
- **OAuth 2.0 and OpenID Connect:** Used for secure token-based authentication and session management.

6.2 Data Encryption

- **Encryption in Transit:** All network communications use TLS 1.3 (RFC 8446) with strong cipher suites (AES-256-GCM, ECDHE key exchange).
- **Encryption at Rest:** Sensitive data including PII and authentication credentials are stored using AES-256 encryption, compliant with NIST FIPS 197. Database columns containing sensitive data utilize field-level encryption.

6.3 Audit and Logging

- **Immutable Audit Trails:** Logs are cryptographically signed and timestamped to prevent tampering.
- **Access Logs:** Include operator ID, IP address, timestamp, and action details.
- **Security Incident Monitoring:** Integration with SIEM tools for real-time anomaly detection.

6.4 Session Management

- **Timeouts:** Sessions automatically expire after configurable periods of inactivity (default 15 minutes).
- **Concurrent Sessions:** Limits set per user to prevent session hijacking.
- **Secure Cookies:** HTTPOnly and Secure flags enabled to safeguard session tokens.

7. Data Formats and Integration

7.1 Data Schema Overview

Data Domain	Format	Protocol	Description
IoT Sensor Data	JSON (RFC 8259)	MQTT over TLS	pH, chlorine, temperature, turbidity readings
Customer Records	JSON / XML	REST API (HTTPS)	Customer profiles, service history
Billing Transactions	JSON	REST API (HTTPS)	Payment status, invoices
Market Intelligence	JSON / CSV	HTTPS	Competitive and environmental data feeds

7.2 Sample IoT Event Payload (JSON)

```
{
  "deviceId": "pool-sensor-001",
  "timestamp": "2024-05-15T14:23:45.123Z",
  "sensorReadings": {
    "pH": 7.4,
    "chlorine_mg_per_l": 2.5,
    "temperature_celsius": 26.1,
    "turbidity_NTU": 1.2
  },
  "status": "normal"
}
```

7.3 API Integration Methods

- **RESTful APIs:** HTTP/HTTPS with JSON payloads, following OpenAPI 3.0 specification for request/response schema validation.

- **WebSocket:** Bidirectional event streams for real-time updates with reconnection and heartbeat protocols implemented to ensure resilience.
- **Batch Data Transfer:** Secure FTP (SFTP) or scheduled API calls for bulk imports/exports.

---

8. Operational Monitoring and Reliability

8.1 Uptime and Availability

- **SLA:** 99.9% uptime, supported by multi-zone redundancy within the DigitalOcean Sydney datacenter.
- **Health Checks:** Automated probes monitor service endpoints, container status, and resource utilization.
- **Failover:** Kubernetes orchestrated pod restarts and node failover ensure resilience.

8.2 Backup and Recovery

- **Backup Schedule:** Daily full and incremental backups stored encrypted with 30-day retention policy.
- **Disaster Recovery:** Tested recovery procedures with RTO (Recovery Time Objective) under 1 hour.
- **Data Integrity:** Backup verification using checksums (SHA-256) to detect corruption.

8.3 Performance Monitoring

- **Metrics Collection:** Prometheus gathers CPU, memory, network, and application metrics.
- **Dashboards:** Grafana provides real-time visualization of system health and alerts.
- **Alerting:** Threshold-based alerts sent to engineering teams via Slack and email.

---

9. Compliance and Legal Considerations

9.1 Australian Privacy Act 1988 Compliance

- **Personal Information Handling:** Customer data is processed in accordance with Australian privacy principles (APPs), ensuring informed consent, purpose limitation, and data minimization.
- **Data Sovereignty:** All data stored within Australian jurisdiction (DigitalOcean Sydney region).
- **Breach Notification:** Procedures in place for timely notification of privacy breaches as mandated by law.

9.2 Intellectual Property

- The Gods Eye Command Centre is proprietary software developed and owned by Palatial Pool Services Pty Ltd.
- Unauthorized use, reproduction, or distribution is prohibited.

---

10. Appendices

Appendix A: Glossary

Term	Definition
WebSocket	A protocol providing full-duplex communication channels over a single TCP connection (RFC 6455).
TLS 1.3	Latest version of Transport Layer Security protocol for encryption (RFC 8446).
AES-256	Advanced Encryption Standard with 256-bit keys, used for data encryption (FIPS 197).
RBAC	Role-Based Access Control for managing user permissions.
TOTP	Time-Based One-Time Password algorithm, used for MFA (RFC 6238).

Appendix B: References

- IETF RFC 6455 – The WebSocket Protocol
- IETF RFC 8446 – Transport Layer Security (TLS) Protocol Version 1.3
- NIST FIPS 197 – Advanced Encryption Standard (AES)
- Australian Privacy Act 1988 – Privacy principles and guidelines
- OAuth 2.0 Authorization Framework (RFC 6749)
- JSON Schema Draft 7 Specification

For further technical inquiries or integration assistance, please contact:
Palatial Pool Services Pty Ltd
Phone: +61 452 157 694
Email: service@palatialpools.com.au

End of Document“`